Midterm Project: QA Implementation and Empirical Analysis

Aldiyar Seylkhanov \ **Alexandr Tyulkov** \ Astana IT University \ Software QA and Testing

Abstract

This report presents the midterm QA analysis for the Circles project, a full-stack TypeScript monorepo containing a React frontend, a Hono backend, and shared utility packages. The goal of the study is to evaluate how the current automation strategy supports risk-based testing and how repository evidence from CI, test execution, and coverage reports can be used to refine quality priorities. The analysis uses direct repository inspection, workflow review, and command-level execution of backend, frontend, and shared-package tests. The observed baseline contains seven automated test files, two CI workflows, and nine files that directly influence test and pipeline behavior. Empirical results show stable backend and shared-package execution, but also reveal a frontend aggregate test failure that does not reproduce when the frontend unit configuration is executed in isolation. This result is important because it shows that partial toolchain unification improves reproducibility while still leaving room for configuration-level interaction faults. The report concludes that the current automation strategy provides strong detectability in a narrow helper-level scope, but broader integration and end-to-end risks remain insufficiently covered.

Introduction

Modern web projects often organize frontend, backend, and shared modules inside one monorepo. This structure improves reuse, but it also increases the number of configuration interactions that affect test execution. Small differences in runners, transforms, environments, or per-package settings can cause tests to behave differently across the same repository. In quality assurance terms, this creates a gap between test existence and test reliability. A project may appear well-automated while still hiding failures that emerge only in combined execution.

The Circles repository provides a suitable case study for this problem. It includes a frontend application, a backend service layer, and shared utilities, all managed inside one TypeScript workspace. Automation is already present through Vitest, Playwright, and GitHub Actions. Earlier assignments established a risk-first strategy and a backend automation baseline. The midterm extends that work by focusing on empirical evidence rather than planned coverage only.

The objective of this report is threefold. First, it documents the current automation architecture and testing scope. Second, it uses observed execution results and coverage data to reassess where quality risk is concentrated. Third, it frames the current repository state as an early technical-report draft that can later evolve into the final paper. The main emphasis is not only on what tests exist, but also on why certain failures appear, what their likely causes are, and what they imply for future QA work.

Literature Review

Existing research on TypeScript and CI provides a useful foundation for this analysis. Bogner and Merkel report that TypeScript projects tend to show better code quality and understandability than JavaScript projects, but do not necessarily exhibit lower bug proneness or faster bug resolution. This is relevant because it suggests that language choice alone does not remove testing and reliability problems. Faults can persist even in typed systems when they arise from configuration, integration, or execution context.

Tang, Alimadadi, and Sumner strengthen this point by identifying tooling, API misuse, and asynchronous handling as common sources of TypeScript defects. Their findings are highly relevant to monorepo QA because many practical failures emerge not from local logic mistakes but from the

interaction between code and its surrounding execution environment. This perspective aligns with the current case, where a frontend unit suite behaves differently depending on how it is executed.

CI literature also supports the importance of infrastructure-level analysis. Wang et al. connect test automation maturity with higher product quality and shorter release cycles. Yu et al. describe CI as an environment composed of tools, metrics, and feedback mechanisms rather than merely a place where tests are run. These findings justify the use of workflow configuration, command structure, and test execution logs as part of QA evidence. Together, the literature suggests that automation quality depends not only on test cases themselves, but also on the consistency of the environment in which those tests execute.

Methodology

Case Study Scope

The evaluation covers three repository areas:

- **Frontend** — React application with unit, browser, and E2E automation.
- **Backend** — Hono and worker logic with helper-level Vitest coverage.
- **Shared utils** — reusable package with a simple Vitest test target.

The study uses only repository-observable evidence. No hypothetical pre-migration state is assumed. Instead, the current repository is analyzed as it exists on 2026-04-10.

Data Collection

Five evidence sources were used:

- repository structure and configuration files;
- package scripts and test locations;
- GitHub Actions workflows;
- direct command execution results;
- coverage outputs and runner warnings.

The following commands were executed:

- `vp run @circles/backend#test`
- `vp run @circles/frontend#test`
- `vp run @circles/backend#test:coverage`
- `vp run @circles/frontend#test:coverage`
- `vp run utils#test:coverage`

Evaluation Dimensions

The empirical analysis is organized around four dimensions:

1. **Configuration surface** — number of files that define testing and CI behavior.
2. **Execution consistency** — whether aggregated and isolated test commands produce matching outcomes.
3. **Coverage and detectability** — what parts of the system are visible to automation.
4. **Pipeline reproducibility** — how repository commands are mapped to CI workflows.

Dimension	Metric	Evidence
Configuration surface	File count	Vite, Vitest, Playwright, and GitHub Actions files
Execution consistency	Pass/fail outcomes	Direct command outputs across packages
Coverage and detectability	Coverage %	V8 coverage reports for instrumented modules
Pipeline reproducibility	Workflow steps	CI YAML and package scripts

Table 1: Evaluation dimensions used in the midterm analysis

Preliminary Results

Automation Inventory

Repository inspection found seven automated test files and two CI workflows. Six of the seven test files belong to unit or component automation, while one is an end-to-end Playwright scenario. The frontend has the largest configuration surface because it uses separate Vite, Vitest, and Playwright configuration files. The backend and shared package each define testing through a single Vite+ configuration file.

Area	Unit/ component files	E2E files	Config count	Main config locations
Frontend	2	1	5	`vite.config.ts`, `vitest.config.ts`, `vitest.unit.config.ts`, `vitest.browser.config.ts`, `playwright.config.ts`
Backend	4	0	1	`apps/backend/ vite.config.ts`
Shared utils	1	0	1	`packages/utils/ vite.config.ts`
CI	N/A	N/A	2	`.github/workflows/ ci.yml`, `.github/ workflows/e2e.yml`

Table 2: Observed automation inventory and configuration surface

Execution Evidence

Backend and shared-package test execution were stable. The backend suite passed with 4 files and 21 tests. The shared package coverage run passed with 1 file and 1 test. The frontend produced the most important unexpected result. The aggregated command `vp run @circles/frontend#test` failed, while the isolated unit command using `vitest.unit.config.ts` passed successfully.

Command	Scope	Files	Tests	Result	Notes
`vp run @circles/backend#test`	Backend	4 passed	21 passed	Pass	Stable helper-level execution
`vp run @circles/frontend#test`	Frontend aggregate	1 passed, 1 failed	2 passed	Fail	Initialization error during multi-project run
`vp test run --config vitest.unit.config.ts`	Frontend unit only	1 passed	4 passed	Pass	Unit scope passes in isolation
`vp run utils#test:coverage`	Shared utils	1 passed	1 passed	Pass	Stable single-package execution

Table 3: Direct execution outcomes

The frontend failure occurred in `src/__tests__/utils.unit.spec.ts` with the message `Cannot read properties of undefined (reading 'config')`. Because the same test passes in isolation, the most likely explanation is not a defect in the tested utility logic, but an issue in test-runner initialization or project composition.

Coverage Evidence

Coverage reports show strong visibility inside the currently instrumented scope:

- backend helper targets: 100% across statements, branches, functions, and lines;
- frontend unit target `src/lib/utils.ts`: 100%;
- shared package target `index.ts`: 100%.

Package	Covered target	Statements	Branches	Functions	Lines
Backend	`range.ts`, `retry.ts`, `spotify-export.ts`, `zip.ts`	100%	100%	100%	100%
Frontend unit	`src/lib/utils.ts`	100%	100%	100%	100%
Shared utils	`index.ts`	100%	100%	100%	100%

Table 4: Observed coverage inside the instrumented scope

These results should be interpreted carefully. Full coverage of a narrow target set does not mean that the system as a whole is well-covered. Controller logic, route-level data behavior, worker orchestration, and most user flows remain outside the measured scope. Detectability is therefore strong in helper modules and weaker in higher-risk integration layers.

Risk Re-evaluation

The empirical evidence supports an updated risk interpretation.

Module / area	Original risk	Observed evidence	Updated risk	Justification
Backend helper layer	High	21 tests passed, 100% targeted coverage	Medium	Likelihood decreases because current helper scope is stable and highly detectable
Frontend test orchestration	Medium	Aggregate run fails while isolated run passes	High	Likelihood increases because combined execution reveals hidden environment interaction
Shared utils	Low	1 test passed, 100% targeted coverage	Low	Current scope is small but stable
Controller and workflow integration	High	No direct coverage evidence in current run	High	Impact remains high and detectability remains low

Table 5: Risk re-evaluation based on current empirical evidence

CI/CD Evidence

The repository currently uses two workflows. The main CI workflow performs checkout, dependency installation, `vp check`, recursive test execution, and recursive build execution on push and pull request events. A second workflow installs Playwright browser dependencies and runs frontend E2E tests. This structure is relatively compact and reproducible, but it also means that frontend reliability depends on both multi-project Vitest execution and a separate Playwright path.

Trigger	→	Install	→	Validation	→	Outcome
Push / PR	→	`pnpm install`	→	`vp check`, `vp run test -r`	→	Build or E2E status

Table 6: Simplified CI execution flow

Primary Interpretation

The main midterm result is that the current QA implementation is strong at the helper level and weaker at the orchestration level. The backend automation baseline is stable, fast, and easy to justify with measurable evidence. The frontend, however, exposes a more complex interaction pattern in which test behavior depends on how the suite is composed. This matters because it shows that the quality of an automation strategy cannot be measured only by coverage percentages or by the existence of test files. Execution context also affects reliability.

Conclusion

The Circles repository already contains a useful QA baseline: automated backend tests, component and utility checks in the frontend, a small shared-package suite, and CI workflows that continuously execute validation steps. However, the midterm evidence also shows that current confidence should remain qualified. While targeted coverage is high, system-wide detectability is still limited, and at least one configuration-level failure emerges only in aggregated frontend execution.

The immediate implication for the next stage of QA work is clear. Future effort should expand beyond helper-level certainty toward controller, route, workflow, and end-to-end behavior. Repeated-run measurement should also be introduced to estimate stability and flakiness rather than relying on one-time observations only. This would make the final report stronger by connecting coverage, execution consistency, and risk reduction more directly.